

# i 筑项目前端开发规范

## JS部分

### 1 代码风格

#### 1.1 文件

[Eslint] [强制] 在文件结尾处，保留一个空行

#### 1.2 结构

##### 1.2.1 缩进

- [Eslint] [强制] 使用两个空格做为一个缩进层级
- [Eslint] [强制] `switch` 下的 `case` 和 `default` 必须增加一个缩进层级

```
// good
switch (variable) {
  case '1':
    // do...
    break
  case '2':
    // do...
    break
  default:
    // do...
}

// bad
switch (variable) {
case '1':
  // do...
  break
case '2':
  // do...
  break
default:
  // do...
}
```

##### 1.2.2 空格

- [Eslint] [强制] 二元运算符两侧必须有一个空格，一元运算符与操作对象之间不允许有空格

```
// good
let a = !arr.length
a++
a = b + c
```

- [Eslint] [强制] 用作代码块起始的左花括号 `{` 前必须有一个空格

```
// good
if (condition) {
}
while (condition) {
}
function funcName() {
}

// bad
if (condition){
}
while (condition){
}
function funcName(){
}
```

- **[Eslint] [强制]** `if` / `else` / `for` / `while` / `function` / `switch` / `do` / `try` / `catch` / `finally` 关键字后，必须有一个空格

```
// good
if (condition) {
}
while (condition) {
}
(function () {
})();

// bad
if(condition) {
}
while(condition) {
}
(function() {
})();
```

- **[Eslint] [强制]** 在对象创建时，属性中的 `:` 之后必须有空格，`:` 之前不允许有空格

```
// good
var obj = {
  a: 1,
  b: 2,
  c: 3,
}

// bad
var obj = {
  a : 1,
  b:2,
  c :3,
}
```

- **[Eslint] [强制]** 函数声明、具名函数表达式、函数调用中，函数名和 `(` 之间不允许有空格。

```
// good
function funcName() {
}
var funcName = function funcName() {
}
funcName()

// bad
function funcName () {
}
var funcName = function funcName () {
}
funcName ()
```

- **[Eslint] [强制]** 在函数调用、函数声明、括号表达式、属性访问、`if / for / while / switch / catch` 等语句中，`()` 和 `[]` 内紧贴括号部分不允许有空格。

```
// good

callFunc(param1, param2, param3)

save(this.list[this.indexes[i]])

needIncream && (variable += increament)

if (num > list.length) {
}

while (len--) {
}

// bad

callFunc( param1, param2, param3 )

save( this.list[ this.indexes[ i ] ] )

needIncreament && ( variable += increament )

if ( num > list.length ) {
}

while ( len-- ) {
}
```

- **[Eslint] [强制]** 行尾不得有多余的空格

### 1.2.3 换行

- **[强制]** 每个独立语句结束后必须换行
- **[强制]** 运算符处换行时，运算符必须在新行的行首

```
// good
if (user.isAuthenticated()
    && user.isInRole('admin')
    && user.hasAuthority('add-admin')
    || user.hasAuthority('delete-admin'))
```

```

) {
    // Code
}
var result = number1 + number2 + number3
    + number4 + number5;

// bad
if (user.isAuthenticated() &&
    user.isInRole('admin') &&
    user.hasAuthority('add-admin') ||
    user.hasAuthority('delete-admin')) {
    // Code
}
var result = number1 + number2 + number3 +
    number4 + number5;

```

- **[强制]** 在函数声明、函数表达式、函数调用、对象创建、数组创建、`for` 语句等场景中，不允许在 `,` 或 `;` 前换行。

```

// good
var obj = {
    a: 1,
    b: 2,
    c: 3
};
foo(
    aVeryVeryLongArgument,
    anotherVeryLongArgument,
    callback
);

// bad
var obj = {
    a: 1
    , b: 2
    , c: 3
};
foo(
    aVeryVeryLongArgument
    , anotherVeryLongArgument
    , callback
);

```

- **[建议]** 不同行为或逻辑的语句，使用空行隔开，更易阅读。

```

// 仅为按逻辑换行的示例，不代表setStyle的最优实现
function setStyle(element, property, value) {
    if (element == null) {
        return;
    }
    // 此处存在一个空行
    element.style[property] = value;
}

```

- **[建议]** 在语句的行长度超过 120 时，根据逻辑条件合理缩进。

// 较复杂的逻辑条件组合，将每个条件独立一行，逻辑运算符放置在行首进行分隔，或将部分逻辑按逻辑组合进行分隔。

// 建议最终将右括号 `)` 与左大括号 `{` 放在独立一行，保证与 ``if`` 内语句块能容易视觉辨识。

```
if (user.isAuthenticated()
    && user.isInRole('admin')
    && user.hasAuthority('add-admin')
    || user.hasAuthority('delete-admin')
) {
    // Code
}
```

// 三元运算符由3部分组成，因此其换行应当根据每个部分的长度不同，形成不同的情况。

```
var result = thisIsAVeryVeryLongCondition
    ? resultA : resultB;
var result = condition
    ? thisIsAVeryVeryLongResult
    : resultB;
```

## 1.2.4 语句

- **[强制]** 在 `if / else / for / do / while` 语句中，即使只有一行，也不得省略块 `{...}`

```
// good
if (condition) {
    callFunc();
}

// bad
if (condition) callFunc();
if (condition)
    callFunc();
```

- **[强制]** `IIFE` 必须在函数表达式外添加 `()`，非 `IIFE` 不得在函数表达式外添加 `()`。

```
// good
var task = (function () {
    // Code
    return result;
})();
var func = function () {
};

// bad
var task = function () {
    // Code
    return result;
}();
var func = (function () {
});
```

## 1.3 命名

Pascal命名法 又名 大驼峰式命名法

Camel命名法 又名 小驼峰式命名法

- **[强制]** 变量 使用 Camel命名法。

```
let loadingModules = {};
```

- **[强制]** 常量 使用 全部字母大写，单词间下划线分隔 的命名方式，力求语义表达完整清楚，不要嫌名字长

```
const HTML_ENTITY = {};
```

- **[强制]** 函数 使用 Camel命名法。方法名命名必须是 动词 或 动词+名词 形式

```
// localValue / getHttpMessage() / inputUserId  
// 特此说明，增删查改，详情统一使用如下 5 个单词，不得使用其他（目的是为了统一各个端）  
add / update / delete / detail / get
```

附： 函数方法常用的动词：

get 获取/set 设置，  
add 增加/remove 删除，  
create 创建/destroy 销毁，  
start 启动/stop 停止，  
open 打开/close 关闭，  
read 读取/write 写入，  
load 载入/save 保存，  
begin 开始/end 结束，  
backup 备份/restore 恢复，  
import 导入/export 导出，  
split 分割/merge 合并，  
inject 注入/extract 提取，  
attach 附着/detach 脱离，  
bind 绑定/separate 分离，  
view 查看/browse 浏览，  
edit 编辑/modify 修改，  
select 选取/mark 标记，  
copy 复制/paste 粘贴，  
undo 撤销/redo 重做，  
insert 插入/delete 移除，  
add 加入/append 添加，  
clean 清理/clear 清除，  
index 索引/sort 排序，  
find 查找/search 搜索，  
increase 增加/decrease 减少，  
play 播放/pause 暂停，  
launch 启动/run 运行，  
compile 编译/execute 执行，  
debug 调试/trace 跟踪，  
observe 观察/listen 监听，  
build 构建/publish 发布，  
input 输入/output 输出，  
encode 编码/decode 解码，

encrypt 加密/decrypt 解密,  
compress 压缩/decompress 解压缩,  
pack 打包/unpack 解包,  
parse 解析/emit 生成,  
connect 连接/disconnect 断开,  
send 发送/receive 接收,  
download 下载/upload 上传,  
refresh 刷新/synchronize 同步,  
update 更新/revert 复原,  
lock 锁定/unlock 解锁,  
check out 签出/check in 签入,  
submit 提交/commit 交付,  
push 推/pull 拉,  
expand 展开/collapse 折叠,  
enter 进入/exit 退出,  
abort 放弃/quit 离开,  
obsolete 废弃/depreciate 废旧,  
collect 收集/aggregate 聚集

```
function stringFormat(source) {  
}
```

- **[强制]** 函数的 参数 使用 `Camel`命名法

```
function hear(theBells) {  
}
```

- **[强制]** 类 / 构造函数 使用 `Pascal`命名法

```
class TextNode {  
}  
  
function TextNode(options) {  
}
```

- **[强制]** 枚举变量 使用 `Pascal`命名法, 枚举的属性 使用 全部字母大写, 单词间下划线分隔 的命名方式。

```
var TargetState = {  
    READING: 1,  
    READED: 2,  
    APPLIED: 3,  
    READY: 4  
};
```

- **[强制]** 由多个单词组成的缩写词, 在命名中, 根据当前命名法和出现的位置, 所有字母的大小写与首字母的大小写保持一致。

```
function XMLParser() {  
}  
function insertHTML(element, html) {  
}  
var httpRequest = new HTTPRequest();
```

- **[建议]** 函数名 使用 动宾短语

```
function getStyle(element) {  
}
```

- **[建议]** `boolean` 类型的变量使用 `is` 或 `has` 开头

```
var isReady = false;  
var hasMoreCommands = false;
```

## 1.4 注释

### 1.4.1 函数/方法注释

- **[强制]** 函数/方法注释必须包含函数说明，有参数和返回值时必须使用注释标识。**[解释]** 当 `return` 关键字仅作退出函数/方法使用时，无须对返回值作注释标识。
- **[强制]** 参数和返回值注释必须包含类型信息，且不允许省略参数的说明。

```
/**  
 * 函数描述  
 *  
 * @param {string} p1 参数1的说明  
 * @param {string} p2 参数2的说明，比较长  
 *      那就换行了。  
 * @param {number=} p3 参数3的说明（可选）  
 * @return {Object} 返回值描述  
 */  
function foo(p1, p2, p3) {  
    var p3 = p3 || 10;  
    return {  
        p1: p1,  
        p2: p2,  
        p3: p3  
    };  
}
```

### 1.4.2 变量，常量，复杂类型注释

- **[建议]** 对变量添加注释。**[强制]** 对重要的变量添加注释
- **[强制]** 对所有的常量添加注释
- **[强制]** 对于嵌套结构的复杂结构的类型，添加注释

```
/**  
 * 服务器信息  
 * @type {[{port: number, host: string},{port: number, host: string}]}  
 * @property {string} host 主机  
 * @property {number} port 端口  
 */  
let servers = [  
    {  
        host: '1.2.3.4',  
        port: 8080,  
    },  
]
```



```
{
  host: '1.2.3.5',
  port: 8081,
},
]
```

### 1.4.3 细节注释

对于内部实现、不容易理解的逻辑说明、摘要信息等，我们可能需要编写细节注释。

- **[建议]** 细节注释遵循单行注释的格式。说明必须换行时，每行是一个单行注释的起始。

```
function foo(p1, p2, p3) {
  // 这里对具体内部逻辑进行说明
  // 说明太长需要换行
  for (...) {
    ....
  }
}
```

- **[强制]** 有时我们会使用一些特殊标记进行说明。特殊标记必须使用单行注释的形式。下面列举了一些常用标记：
  - **TODO**: 有功能待实现。此时需要对将要实现的功能进行简单说明。
  - **FIXME**: 该处代码运行没问题，但可能由于时间赶或者其他原因，需要修正。此时需要对如何修正进行简单说明。
  - **HACK**: 为修正某些问题而写的不太好或者使用了某些诡异手段的代码。此时需要对思路或诡异手段进行描述。
  - **xxx**: 该处存在陷阱。此时需要对陷阱进行描述。

## 2 语言特性

### 2.1 变量

- **[强制]** 不可使用 `var` 关键字声明变量
- **[强制]** 变量必须 即用即声明，不得在函数或其它形式的代码块起始位置统一声明所有变量。**[解释]**: 变量声明与使用的距离越远，出现的跨度越大，代码的阅读与维护成本越高。虽然JavaScript的变量是函数作用域，还是应该根据编程中的意图，缩小变量出现的距离空间

```
// good
function kv2List(source) {
  var list = [];
  for (var key in source) {
    if (source.hasOwnProperty(key)) {
      var item = {
        k: key,
        v: source[key]
      };
      list.push(item);
    }
  }
  return list;
}

// bad
function kv2List(source) {
  var list = [];
```

```

var key;
var item;
for (key in source) {
    if (source.hasOwnProperty(key)) {
        item = {
            k: key,
            v: source[key]
        };
        list.push(item);
    }
}
return list;
}

```

## 2.2 条件

- **[强制]** 在 Equality Expression 中使用类型严格的 `===`。仅当判断 `null` 或 `undefined` 时，允许使用 `== null`。**[解释]** 使用 `===` 可以避免等于判断中隐式的类型转换。

```

// good
if (age === 30) {
    // .....
}
// bad
if (age == 30) {
    // .....
}

```

- **[建议]** 尽可能使用简洁的表达式。

```

// 字符串为空
// good
if (!name) {
    // .....
}
// bad
if (name === '') {
    // .....
}

```

```

// 字符串非空
// good
if (name) {
    // .....
}
// bad
if (name !== '') {
    // .....
}

```

```
// 数组非空
// good
if (collection.length) {
    // .....
}
// bad
if (collection.length > 0) {
    // .....
}
```

```
// 布尔不成立
// good
if (!notTrue) {
    // .....
}
// bad
if (notTrue === false) {
    // .....
}
```

```
// null 或 undefined
// good
if (noValue == null) {
    // .....
}
// bad
if (noValue === null || typeof noValue === 'undefined') {
    // .....
}
```

- **[建议]** 按执行频率排列分支的顺序。
  - **解释：**按执行频率排列分支的顺序好处是：
    - 1、阅读的人容易找到最常见的情况，增加可读性。
    - 2、提高执行效率。
- **[建议]** 对于相同变量或表达式的多值条件，用 `switch` 代替 `if`。

```
// good
switch (typeof variable) {
    case 'object':
        // .....
        break;
    case 'number':
    case 'boolean':
    case 'string':
        // .....利用了case穿透
        break;
}
// bad
var type = typeof variable;
if (type === 'object') {
    // .....
}
else if (type === 'number' || type === 'boolean' || type === 'string') {
    // .....
}
```

```
}
```

- **[建议]** 如果函数或全局中的 `else` 块后没有任何语句，可以删除 `else`。  
示例：

```
// good
function getName() {
  if (name) {
    return name;
  }
  return 'unnamed';
}

// bad
function getName() {
  if (name) {
    return name;
  }
  else {
    return 'unnamed';
  }
}
```

## 2.3 循环

- **[建议]** 对循环内多次使用的不变值，在循环外用变量缓存。

```
// good
var width = wrap.offsetWidth + 'px';
for (var i = 0, len = elements.length; i < len; i++) {
  var element = elements[i];
  element.style.width = width;
  // .....
}

// bad
for (var i = 0, len = elements.length; i < len; i++) {
  var element = elements[i];
  element.style.width = wrap.offsetWidth + 'px';
  // .....
}
```

## 2.4 类型

- **[建议]** 转换成 `string` 时，使用 `+` 和 `''`。

```
// good
num + '';

// bad
new String(num);
num.toString();
String(num);
```

- **[建议]** `string` 转换成 `number`，要转换的字符串结尾包含非数字并期望忽略时，使用 `parseInt`。

```
var width = '200px';
parseInt(width, 10);
```

- **[强制]** 使用 `parseInt` 时，必须指定进制。

```
// good
parseInt(str, 10);
// bad
parseInt(str);
```

- **[建议]** 转换成 `boolean` 时，使用 `!!`。

```
var num = 3.14;
!!num;
```

- **[建议]** `number` 去除小数点，使用 `Math.floor` / `Math.round` / `Math.ceil`，不使用 `parseInt`。

```
// good
var num = 3.14;
Math.ceil(num);
// bad
var num = 3.14;
parseInt(num, 10);
```

## 2.5 字符串

- **[强制]** 字符串和变量拼接，使用 模板字符串

## 2.6 对象

- **[强制]** 使用对象字面量 `{}` 创建新 `Object`

```
// good
var obj = {};
// bad
var obj = new Object();
```

- **[建议]** 属性访问时，尽量使用 `.`。**[解释]**: 属性名符合 `Identifier` 的要求，就可以通过 `.` 来访问，否则就只能通过 `[expr]` 方式访问。  
通常在 JavaScript 中声明的对象，属性命名是使用 Camel 命名法，用 `.` 来访问更清晰简洁。部分特殊的属性（比如来自后端的 JSON），可能采用不寻常的命名方式，可以通过 `[expr]` 方式访问。  
示例：

```
var info = {
  name: 'someone',
  age: 28,
  'more-info': '...',
};
info.age;
info['more-info'];
```

- **[建议]** `for in` 遍历对象时, 使用 `hasOwnProperty` 过滤掉原型中的属性。

```
var newInfo = {};  
for (var key in info) {  
  if (info.hasOwnProperty(key)) {  
    newInfo[key] = info[key];  
  }  
}
```

## 2.7 使用 ES6+

- 必须优先使用 ES6+ 中新增的语法糖和函数, 这将简化程序。比如箭头函数、`async/await`、解构、`const`、`for...of`、模板字符串 等

## CSS部分

- **[强制]** `css` 中 `class` 必须单词全字母小写, 单词间以 `-` 分隔。 `class` 必须代表相应模块或部件的内容或功能, 不得以样式信息进行命名
- **[强制]** 颜色值不允许使用命名色值
- **[强制]** `font-weight` 属性必须使用数值方式描述
- **[强制]** 选择器 与 `{` 之间必须包含空格。

```
.selector {  
}
```

- **[强制]** 属性名 与之后的 `:` 之间不允许包含空格, `:` 与 属性值 之间必须包含空格。

```
margin: 0;
```

- **[建议]** 非必要不定义 `id`, 而使用 `vue` 提供的 `ref` 属性, 采用 小驼峰式 命名
- **[建议]** 使用 `border` / `margin` / `padding` 等缩写时, 应注意隐含值对实际数值的影响, 确实需要设置多个方向的值时才使用缩写
- **[建议]** 尽量不使用 `!important` 声明
- **[强制]** 长度为 `0` 时须省略单位

```
/* good */  
body {  
  padding: 0 5px;  
}  
  
/* bad */  
body {  
  padding: 0px 5px;  
}
```

## Vue项目

### 命名规范

- 目录和文件 (`JS`、`CSS`、`LESS`、`SVG`、`PNG`、`VUE`) 命名: 全部采用小写方式, 以中划线分割
- 组件名应该始终是多单词组成 (大于等于 2)

```
// good

components/
|- my-component.vue

// good
frontend/src/views/home/components/content-area.vue

// bad
frontend/src/views/chat/Chat.vue
```

- 命名要望文知义，杜绝不规范的缩写

## Vue Template

- 缩进：嵌套的节点应该缩进，同 js
- HTML5 中新增很多语义化标签，所以优先使用语义化标签，避免一个页面都是 div 或者 p 标签

```
<header></header>
<footer></footer>
```

- 使用双引号 `"`，而不是单引号 `'`
- 在 Template 模版中使用组件，应使用 PascalCase 模式，并且使用自闭合组件

```
<!-- 在单文件组件-->
<MyComponent />
```

## Vue Script

### Prop 定义应该尽量详细

- 必须使用 camelCase 驼峰命名
- 必须指定类型必须加上注释，表明其含义
- 必须加上 required 或者 default，两者二选其一
- 如果有业务需要，必须加上 validator 验证

```
props: {
  // 组件状态，用于控制组件的颜色
  status: {
    type: String,
    required: true,
    validator: function (value) {
      return [
        'succ',
        'info',
        'error'
      ].indexOf(value) !== -1
    }
  },
  // 用户级别，用于显示皇冠个数
  userLevel: {
    type: String,
    required: true
  }
}
```

```
}
```

## 为组件样式设置作用域

```
<template>
  <button class="btn btn-close">X</button>
</template>
<!-- 使用 `scoped` 特性 -->
<style scoped>
  .btn-close {
    background-color: red;
  }
</style>
```

## 模板中使用简单的表达式

- 组件模板应该只包含简单的表达式
- 复杂的表达式则应该重构为计算属性或方法。复杂表达式会让你的模板变得不那么声明式。我们应该尽量描述应该出现的是什麼，而非如何计算那个值。而且计算属性和方法使得代码可以重用。

```
// good

<template>
  <p>{{ normalizedFullName }}</p>
</template>

// 复杂表达式已经移入一个计算属性
computed: {
  normalizedFullName: function () {
    return this.fullName.split(' ').map(function (word) {
      return word[0].toUpperCase() + word.slice(1)
    }).join(' ')
  }
}

// bad

<template>
  <p>
    {{
      fullName.split(' ').map(function (word) {
        return word[0].toUpperCase() + word.slice(1)
      }).join(' ')
    }}
  </p>
</template>
```

## 标签顺序保持一致

单文件组件应该总是让标签顺序保持为

```
<template>...</template>
<script>...</script>
<style>...</style>
```



必须为 v-for 设置键值 key

使用路由懒加载（延迟加载）机制

函数参数控制在 3 个（含）以内，参数过多情况下，可使用 对象 传参

## 项目部分

---

代码提交前，项目可运行

多个无效commit，可使用 rebase 合并

git 提交名字更改

## 规范问题，根据实际项目情况做更改和补充

---